SERFIT

Documentação do projeto

Academia SERFIT · Unidade ao lado da Jaqueira · 23 de outubro de 2026

Site e sistema da academia **SERFIT**, unidade única ao lado da Jaqueira.

Reúne a página de apresentação da academia e três áreas com login: **aluno**, **professor** e **gestão**. Login, cadastro, avisos e cobranças ficam guardados em um banco SQLite.

Funcionamento da academia

Dia	Horário
Segunda a sexta	05:00 – 22:00
Sábado	05:00 – 12:00
Domingo	Fechado

São 4 professores, 2 no turno da manhã (05:00 – 13:00) e 2 no turno da tarde (13:00 – 22:00).

Como rodar

Pré-requisito: **Node.js 22.13 ou mais novo**. O SQLite já vem embutido no Node (`node:sqlite`), então não é preciso instalar banco de dados.

```
npm install
npm start
```

Abra **http://localhost:3000**.

- Para usar outra porta no PowerShell: `$env:PORT=3001; npm start`
- Depois de mudar `server.js`, `db.js` ou `seed.js`, pare o servidor (Ctrl + C) e rode `npm start` de novo. Para mudanças em `public/`, basta recarregar a página.

Primeiro acesso

Na primeira vez que o servidor sobe, ele cria o banco em `data/serfit.db` com:

- o usuário **GESTAO** (master), com a senha definida em `seed.js`;
- dois avisos de exemplo.

Alunos e professores se cadastram pelo próprio site. **Troque a senha da gestão antes de colocar o site no ar**, porque ela está escrita no repositório.

Perfis e permissões

Perfil	Como a conta é criada	O que pode fazer
Aluno	Cadastro no site, entra na hora	Ver avisos; escolher ou trocar professor e plano; ver a mensalidade e o histórico; reservar aulas de zumba
Professor	Cadastro no site, **aguarda aprovação**	Ver os alunos que o escolheram; montar treino; publicar avisos
Professor master	Professor marcado como master no banco	Tudo do professor, mais aprovar ou recusar novos professores
Gestão (master)	Criada pelo `seed.js`	Aprovar ou recusar professores; ver cobranças por mês e marcar como paga; publicar avisos; controlar o estacionamento

Regras

- **Cadastro**: nome de 3 a 30 letras (acentos são aceitos, números e símbolos não); senha de 6 a 72 caracteres. O nome não diferencia maiúsculas de minúsculas e não pode se repetir.
- **Professor novo** fica com status `pendente` e não consegue entrar até um master aprovar. Recusar apaga o cadastro.
- **Aluno novo** escolhe o plano no cadastro e começa **sem professor**. Ele escolhe o professor depois, no painel.
- **Limite de tentativas**: 5 logins errados seguidos bloqueiam aquele nome por 5 minutos; o limite para cadastros é de 5 a cada 5 minutos por conexão.
- **Sessão**: dura 8 horas, guardada em cookie `HttpOnly`.

Planos e mensalidades

Plano	Código	Valor mensal
Musculação	`musculacao`	R$ 120,00
Musculação e dança	`musculacao_danca`	R$ 160,00

Os valores ficam em `PLANOS`, em `db.js`.

- A mensalidade do mês é lançada como **pendente** quando o aluno se cadastra.
- O servidor lança automaticamente a mensalidade do mês para todo aluno com plano que ainda não tenha. Isso acontece ao iniciar e sempre que um aluno ou a gestão abre as cobranças, de modo que um mês novo aparece sozinho.
- Quando o aluno troca de plano, a mensalidade do mês muda de valor **se ainda não foi paga**. Se já foi paga, o novo valor só vale para o mês seguinte.
- A gestão marca cada mensalidade como paga (ou desfaz). A situação pode ser `paga`, `pendente` ou `atrasada`.

Estrutura do projeto

```
SERFIT/
├── server.js        Servidor Express: rotas da API e arquivos do site
├── db.js            Banco SQLite: tabelas, migrações e consultas
├── seed.js          Dados iniciais (usuário GESTAO e avisos)
├── package.json
├── public/          Site (servido como arquivos estáticos)
│   ├── index.html   Todas as telas: início, login, aluno, professor, gestão
│   ├── script.js    Lógica das telas e chamadas à API
│   └── style.css
└── data/            Banco serfit.db (criado ao rodar; fora do Git)
```

O site é uma página única: as telas são seções do `index.html`, e o `script.js` mostra uma de cada vez com `mostrarPagina(id)`.

Banco de dados

Arquivo: `data/serfit.db`. As tabelas são criadas pelo `db.js` ao iniciar, e colunas novas são adicionadas automaticamente em bancos antigos.

Tabela	Conteúdo
`usuarios`	`nome` (único), `senha_hash`, `tipo` (`aluno`, `professor`, `gestao`), `status` (`ativo`, `pendente`), `turno` (`manha`, `tarde`), `plano`, `professor_id` (professor escolhido pelo aluno), `master`
`sessoes`	Token de login, usuário e validade
`avisos`	Texto, autor e data
`cobrancas`	Aluno, mês (`AAAA-MM`), valor em centavos, situação e data de pagamento. Uma por aluno por mês

As senhas são guardadas com `scrypt` e um salt aleatório, nunca em texto aberto. Os valores ficam em **centavos** para evitar erro de arredondamento.

API

Todas as rotas respondem em JSON. As que exigem login usam o cookie da sessão.

Método	Rota	Quem pode	O que faz
POST	`/api/login`	Todos	Entra com `{ nome, senha }`
POST	`/api/cadastro`	Todos	Cria conta com `{ nome, senha, tipo, plano, turno }` (plano para aluno, turno para professor)
POST	`/api/logout`	Todos	Sai e apaga a sessão
GET	`/api/me`	Logado	Dados de quem está logado
GET	`/api/planos`	Todos	Planos e valores
GET	`/api/avisos`	Todos	Últimos avisos

Método	Rota	Quem pode	O que faz
POST	`/api/avisos`	Professor, gestão	Publica aviso `{ texto }` (até 500 caracteres)
GET	`/api/aluno/perfil`	Aluno	Plano e professor do aluno
GET	`/api/professores`	Aluno	Professores ativos para escolher
PUT	`/api/aluno/professor`	Aluno	Escolhe professor `{ professorId }` (`null` = sem professor)
PUT	`/api/aluno/plano`	Aluno	Troca o plano `{ plano }`
GET	`/api/cobrancas/minhas`	Aluno	Mensalidades do aluno
GET	`/api/professor/alunos`	Professor	Alunos que escolheram este professor
GET	`/api/professores/pendentes`	Master	Professores aguardando aprovação
POST	`/api/professores/:id/aprovar`	Master	Aprova o professor
DELETE	`/api/professores/:id`	Master	Recusa e apaga o cadastro pendente
GET	`/api/cobrancas/meses`	Gestão	Meses que têm cobranças
GET	`/api/cobrancas?mes=AAAA-MM`	Gestão	Cobranças do mês
PATCH	`/api/cobrancas/:id`	Gestão	Muda a situação `{ situacao }`

"Master" significa gestão ou professor master.

O que ainda é demonstrativo

Estas partes aparecem no site, mas ainda **não são salvas no banco**. Elas usam dados fixos do `script.js` e se perdem ao recarregar:

- **Equipe** na página inicial: nomes de exemplo, não os professores cadastrados.
- **Treinos**: o professor edita os exercícios, mas eles não ficam salvos.
- **Zumba**: grade e reservas.
- **Estacionamento**: entradas, saídas e o gráfico por horário.

Próximos passos

- Salvar treinos, zumba e estacionamento no banco.
- Tela para a gestão marcar um professor como master. Hoje só existe no banco, pela coluna `master`.
- Tela para trocar a senha.
- Esconder os painéis no HTML até o login ser confirmado. Hoje os dados reais já são protegidos pela API, mas a estrutura das telas fica visível no código da página.
- Pagamento online (Pix, boleto, cartão).